

34. Kombinační logické obvody (TAHÁK)

Klíčové pojmy

Kombinační logický obvod, Logické hradlo, Booleova algebra, Pravdivostní tabulka, Minterm, Maxterm, Úplná disjunktivní normální forma (UDNF), Úplná konjunktivní normální forma (UKNF), Karnaughova mapa (K-map), Propagační zpoždění, Univerzální logický člen, Multiplexor (MUX), Demultiplexor (DEMUX), Dekodér (DEC), Kodér (ENC), Sčítačka, Komparátor.

A. Charakteristika kombinačních logických obvodů

Digitální obvod, jehož výstupní hodnoty závisí **výhradně na aktuálních hodnotách vstupů**. Nemá vnitřní paměť ani zpětnou vazbu.

Hlavní rysy: * **Bezpečnost:** Výstupy jsou funkcí pouze aktuálních vstupů. *

Absence zpětné vazby: Žádné smyčky vracející výstupy na vstupy. * **Matematický popis:** Booleovy funkce nebo pravdivostní tabulka. *

Propagační zpoždění: Nenulový čas mezi změnou vstupu a výstupem. * **Příklady:** Sčítačka, Multiplexor, Demultiplexor, Kodér, Dekodér, Komparátor.

Rozdíl od sekvenčních obvodů: Sekvenční obvody (klopné obvody, registry, čítače) mají paměť – výstup závisí i na předchozím stavu.

B. Realizace pomocí logických hradel

Logická hradla realizují elementární Booleovy operace.

Základní logická hradla: * **AND:** $Y = A \cdot B$ (výstup 1, jen když všechny vstupy 1) *

OR: $Y = A + B$ (výstup 1, když alespoň jeden vstup 1) * **NOT:** $Y = \overline{A}$ (výstup opačný než vstup) * **NAND:** $Y = \overline{A \cdot B}$ (výstup 0, jen když všechny vstupy 1) * **NOR:** $Y = \overline{A + B}$ (výstup 1, jen když všechny vstupy 0) *

XOR: $Y = A \oplus B = A \cdot \overline{B} + \overline{A} \cdot B$ (výstup 1, když vstupy různé) * **XNOR:**

$Y = \overline{A \oplus B} = A \cdot B + \overline{A} \cdot \overline{B}$ (výstup 1, když vstupy stejné)

Postup návrhu obvodu z hradel: 1. **Pravdivostní tabulka:** Určení výstupů pro všechny kombinace vstupů. 2. **Booleova funkce (UDNF):** Součet mintermů (řádky s výstupem 1). Minterm je součin vstupů, negovaných pro hodnotu 0. * **Příklad XOR:** $Y = \overline{A} \cdot B + A \cdot \overline{B}$ 3. **Minimalizace funkce:** Zjednodušení pro snížení počtu hradel, nákladů, spotřeby a zpoždění. * **Metody:** Booleova algebra, Karnaughova mapa (pro

2-5 proměnných). 4. **Nakreslení schématu:** Realizace zjednodušené funkce pomocí hradel.

Univerzálnost hradel NAND / NOR: Pomocí samotných hradel NAND (nebo samotných NOR) lze realizovat jakoukoliv logickou funkci (NOT, AND, OR, XOR). *

Příklad NOT z NAND: $A \text{ NAND } A = \overline{A \cdot A} = \overline{A}$

C. Realizace pomocí Multiplexorů (MUX)

MUX: Digitální přepínač s 2^n datovými vstupy (D_i), n selekčními vstupy (S_j) a jedním výstupem (Y). Výstup Y je roven datovému vstupu, jehož index odpovídá binární hodnotě na selekčních vstupech. * *Funkce MUX 4:1:*

$$Y = \overline{S_1} \cdot \overline{S_0} \cdot D_0 + \overline{S_1} \cdot S_0 \cdot D_1 + S_1 \cdot \overline{S_0} \cdot D_2 + S_1 \cdot S_0 \cdot D_3$$

Realizace libovolné logické funkce pomocí MUX: 1. **Proměnné na selekční**

vstupy: n proměnných funkce na n selekčních vstupů MUX (2^n datových vstupů). 2.

Logické hodnoty na datové vstupy: Na D_0 až D_{2^n-1} se přivede 0 nebo 1 dle pravdivostní tabulky funkce pro danou kombinaci selekčních vstupů. * *Příklad XOR (MUX 4:1, $S_1 = A, S_0 = B$): $D_0 = 0, D_1 = 1, D_2 = 1, D_3 = 0$.*

D. Realizace pomocí Dekodérů (DEC)

Dekodér: Obvod s n vstupy a 2^n výstupy. Aktivuje právě jeden výstup (Y_k), jehož index odpovídá binárnímu číslu na vstupech. Každý výstup Y_k odpovídá jednomu mintermu. * *Příklad DEC 2:4 (A_1, A_0): $Y_0 = \overline{A_1} \cdot \overline{A_0}, Y_1 = \overline{A_1} \cdot A_0, Y_2 = A_1 \cdot \overline{A_0}, Y_3 = A_1 \cdot A_0$.*

Realizace libovolné logické funkce pomocí DEC: 1. **Vstupy na dekodér:** n

proměnných funkce na vstupy dekodéru (2^n výstupů). 2. **Identifikace mintermů:**

Najít řádky v pravdivostní tabulce, kde je výstup funkce 1. 3. **Zapojení hradla OR:**

Výstupy dekodéru odpovídající těmto mintermům se zapojí do vícevstupového

hradla OR. Výstup OR je výsledná funkce. * *Příklad XOR (DEC 2:4, A, B): Výstup 1 pro $AB = 01$ (Y_1) a $AB = 10$ (Y_2). Funkce je $Y = Y_1 + Y_2$.*

Shrnutí možností realizace

Hradla: Nejzákladnější, flexibilní pro jednoduché funkce. Vyžaduje minimalizaci (K-mapy, Booleova algebra).

Multiplexor: Univerzální, přímé mapování pravdivostní tabulky na datové vstupy. Bez minimalizace. Pro mnoho vstupů může být velký.

Dekodér: Generuje všechny mintermy. Ideální pro více funkcí z jedněch vstupů (s přidáním OR hradel). Pro jednu funkci méně efektivní než MUX.

Praktické použití

ALU (sčítačky, komparátory), Adresování paměti, Datové směrování (MUX/DEMUX), Řízení displejů, Kódování/dekódování, Řadiče.

Nejčastější chyby u zkoušky

Přidání paměti do kombinačního obvodu (záměna se sekvenčním).

Chyby v De Morganových zákonech.

Neúplná pravdivostní tabulka.

Neznalost univerzálnosti NAND/NOR hradel.

Nepochopení principu MUX/DEC pro realizaci funkcí.

Typické zkouškové otázky (zkrácené odpovědi)

Co je kombinační obvod a jak se liší od sekvenčního obvodu? Kombinační obvod:

Výstup závisí **jen na aktuálních vstupech**, nemá paměť ani zpětnou vazbu.

Sekvenční obvod: Výstup závisí na aktuálních vstupech **i na předchozím stavu** (má paměť), často řízen hodinovým signálem.

Popište základní logická hradla a uveďte jejich Booleovy funkce a pravdivostní tabulky. Základní hradla: AND ($Y = A \cdot B$), OR ($Y = A + B$), NOT ($Y = \overline{A}$), NAND ($Y = \overline{A \cdot B}$), NOR ($Y = \overline{A + B}$), XOR ($Y = A \oplus B$), XNOR ($Y = \overline{A \oplus B}$). Pro každé hradlo je nutné uvést jeho pravdivostní tabulku.

Jak se navrhuje kombinační obvod z pravdivostní tabulky a jakou roli hraje minimalizace? Návrh: 1. Pravdivostní tabulka. 2. Zápis Booleovy funkce (UDNF z mintermů). 3. **Minimalizace** (Booleova algebra, K-mapy) – klíčová pro zjednodušení obvodu, snížení hradel, nákladů, spotřeby a zpoždění. 4. Nakreslení schématu.

Vysvětlete, jak lze libovolnou logickou funkci realizovat pomocí multiplexoru nebo dekodéru.

Multiplexor (MUX): Vstupní proměnné funkce na selekční vstupy MUX. Na datové vstupy MUX se přivedou pevné logické hodnoty (0/1) odpovídající výstupům funkce z pravdivostní tabulky.

Dekodér (DEC): Vstupní proměnné funkce na vstupy dekodéru. Výstupy dekodéru, které odpovídají mintermům s výstupem 1 v pravdivostní tabulce, se zapojí do vícevstupového hradla OR.

35. Sekvenční logické obvody (TAHÁK)

Klíčové pojmy

Sekvenční obvod, Klopný obvod (Flip-Flop, Latch), Synchronní/Asynchronní, Hodiny (CLK), Registry, Posuvný registr, Čítač, Konečný automat (FSM), Moore, Mealy, Graf stavů, Tabulka přechodů, Kódování stavů.

A. Základní vlastnosti sekvenčních obvodů

Digitální obvod, jehož výstup závisí na aktuálních vstupech **a předchozím vnitřním stavu**. * **Paměť**: Obsahuje paměťové prvky (klopné obvody) pro uchování stavu. *

Zpětná vazba: Výstupy paměťových prvků ovlivňují budoucí stav.

Rozdělení podle synchronizace:

Asynchronní: Reagují okamžitě na změnu vstupů. Náchylné na hazardy.

Synchronní (řízené hodinami): Změna stavu jen na definované hraně CLK .

Koordinované, standard.

B. Typy klopných obvodů (Flip-Flops / Latches)

Základní jednobitový paměťový prvek (Q , $\bar{Q}$).

RS (Reset-Set):

Vstupy: S , R .

Chování: $S = 1, R = 0 \rightarrow Q = 1$; $S = 0, R = 1 \rightarrow Q = 0$; $S = 0, R = 0 \rightarrow Q_{prev}$.

Zakázaný stav: $S = 1, R = 1$.

Char. rovnice (pro $SR \neq 11$): $Q_{next} = S + \bar{R}Q_{prev}$

D (Data / Delay):

Vstupy: D , CLK .

Chování: Na aktivní hraně CLK převezme D na výstup Q .

Char. rovnice: $Q_{next} = D$. Základní prvek registrů.

JK:

Vstupy: J , K , CLK .

Chování (na aktivní hraně CLK):

$J = 0, K = 0 \rightarrow Q_{prev}$

$J = 0, K = 1 \rightarrow Q = 0$

$J = 1, K = 0 \rightarrow Q = 1$

$J = 1, K = 1 \rightarrow \bar{Q}_{prev}$

Char. rovnice: $Q_{next} = J\bar{Q}_{prev} + \bar{K}Q_{prev}$

T (Toggle):

Vstupy: T, CLK .

Chování (na aktivní hraně CLK):

$$T = 0 \rightarrow Q_{prev}$$

$$T = 1 \rightarrow \bar{Q}_{prev}$$

Char. rovnice: $Q_{next} = T \oplus Q_{prev}$. Základ čítačů.

C. Registry a posuvné registry

Sdružení klopných obvodů (typicky D) pro uchování vícebitového slova.

Paralelní registr (Buffer): Společný CLK . Data zapisována/čtena paralelně (PIPO).

Posuvný registr (Shift): Klopné obvody sériově. S každým CLK se info posune.

Využití: Převod sériový/paralelní (SIPO/PISO), mat. operace ($\cdot 2, /2$).

D. Čítače (Counters)

Sekvenční obvody pro počítání hodinových impulsů. Výstup inkrementuje/dekrementuje.

Asynchronní (Ripple): Výstup FF_i budí $CLK_{FF_{i+1}}$. Sčítá se zpoždění, nevhodné pro vysoké frekvence.

Synchronní: Všechny FF řízeny společným CLK . Změna stavu všech bitů současně. Řízeno kombinační logikou.

E. Návrh konečných automatů (FSM)

Složitější sekvenční obvody (řadiče). Popsán: vstupy, výstupy, stavy, přechodová funkce.

Automat typu Moore: Výstup je funkcí **pouze aktuálního stavu**. Stabilní výstupy, výstup ve stavu (kroužku).

Automat typu Mealy: Výstup je funkcí aktuálního stavu **i aktuálních vstupů**.

Okamžitá reakce na vstup, výstup na přechodu (šipce). Méně stavů, náchylnější na glitche.

Postup návrhu synchronního automatu:

Graf stavů / Tabulka přechodů: Specifikace chování.

Kódování stavů: Binární kód pro každý stav ($\lceil \log_2 N \rceil$ FF pro N stavů).

Zakódovaná tabulka přechodů a výstupů: $Q_{actual}, \text{vstupy} \rightarrow Q_{next}, \text{výstupy}$.

Minimalizace kombinační logiky: Karnaughovy mapy pro buzení FF a výstupy.

Realizace: Zapojení hradel a FF (společný CLK).

Praktické použití

Paměťové prvky CPU, Časovače/Čítače, Řadiče protokolů (UART, SPI), FSM v FPGA, Řízení sekvencí.

Nejčastější chyby u zkoušky

Zaměňování kombinačního/sekvenčního.

Ignorování hodin/synchronizace.

Neřešení počátečního stavu.

Neznalost zakázaných stavů (RS).

Nerozlišování Moore/Mealy.

Typické zkouškové otázky

Co je sekvenční obvod?

Výstup závisí na aktuálních vstupech a předchozím stavu. Obsahuje paměť (FF) a zpětnou vazbu.

Jak funguje registr a čítač?

Registr: Skupina FF pro uchování vícebitového slova. Společný *CLK*. Paralelní (PIPO) nebo posuvný (sériový posun).

Čítač: Počítá *CLK* impulsy, výstup inkrementuje. Asynchronní (řetězové zpoždění) nebo synchronní (společný *CLK*, řízený logikou).

Jak se popisuje konečný automat?

Množiny: vstupy, výstupy, stavy. Funkce: přechodová (stav, vstup $\rightarrow$ nový stav) a výstupní (stav $\rightarrow$ výstup (Moore); stav, vstup $\rightarrow$ výstup (Mealy)). Graficky: graf stavů; Formálně: tabulka přechodů.

36. Aritmetické obvody (TAHÁK)

Klíčové pojmy: binární kód, dvojkový doplněk, znaménkový bit, MSB, LSB, poloviční sčítačka (HA), úplná sčítačka (FA), sčítačka s řetězovým přenosem (RCA), sčítačka s předpovědí přenosu (CLA), násobička, sériová násobička, paralelní násobička, přetečení (Overflow), přenos (Carry), výpůjčka (Borrow).

A. Reprezentace čísel v počítači

Neznaménková čísla (Unsigned): Přímý binární kód. Rozsah pro n bitů: 0 až $2^n - 1$.

Znaménková čísla (Signed):

Přímý kód: MSB = znaménko ($0 = +$, $1 = -$). Nevýhoda: Dvě reprezentace nuly ($+0$, -0).

Inverzní kód (Jedničkový doplněk): Záporné = negace všech bitů kladného. Nevýhoda: Dvě nuly.

Doplňkový kód (Dvojkový doplněk): Dominantní. Záporné = negace všech bitů kladného + 1.

Výhody: Jedna reprezentace nuly, sčítání i odčítání pomocí téže sčítačky. Rozsah pro n bitů: -2^{n-1} až $2^{n-1} - 1$.

B. Sčítačky (Adders)

1. Poloviční sčítačka (HA - Half Adder)

Sčítá A, B . Generuje součet S a přenos C . * **Rovnice:** $S = A \oplus B$, $C = A \cdot B$. *

Omezení: Nelze zpracovat přenos C_{in} z nižšího řádu.

2. Úplná sčítačka (FA - Full Adder)

Sčítá A, B, C_{in} . Generuje součet S a přenos C_{out} . * **Rovnice:** $S = A \oplus B \oplus C_{in}$, $C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$.

3. Vícebitové sčítačky

Sčítačka s řetězovým přenosem (RCA):

Konstrukce: Sériové zapojení n FA. C_{out} jedné FA je C_{in} další.

Nevýhoda: Pomalá, zpoždění přenosu se šíří postupně.

Časová složitost: $O(n)$.

Sčítačka s předpovědí přenosu (CLA):

Konstrukce: Speciální logika paralelně "předpoví" přenosy pro všechny bity.

Výhoda: Extrémně rychlá, eliminuje řetězové zpoždění.

Nevýhoda: Složitá struktura, vysoká hardwarová náročnost.

Časová složitost: $O(\log n)$ nebo téměř $O(1)$ pro bloky.

C. Odčítačky a doplňkový kód

Odčítání $A - B$ se realizuje jako sčítání $A + (-B)$. * **Rovnice:** $A - B = A + \bar{B} + 1$ (kde $\bar{B}$ je bitová negace B). * **Realizace:** * Vstup A přímo. * Vstup B přes XOR hradla řízená signálem `SUB`. * Signál `SUB` je zároveň C_{in} pro nejnižší bit. * `SUB = 0` (sčítání): $B \oplus 0 = B, C_{in} = 0 \implies A + B$. * `SUB = 1` (odčítání): $B \oplus 1 = \bar{B}, C_{in} = 1 \implies A + \bar{B} + 1$.

D. Násobičky (Multipliers)

Násobení = generování částečných součinů a jejich sčítání s posunem. * **Sériová násobička:** * **Princip:** Algoritmus posunu a sčítání. Jeden registr, posuvný registr, jedna sčítačka. * **Výhoda:** Prostorově nenáročná. * **Nevýhoda:** Pomalá (n taktů pro n -bitové násobení). * **Paralelní (Pole) násobička:** * **Konstrukce:** Matice AND hradel (částečné součiny) a sít' sčítaček (např. Wallaceův strom). * **Výhoda:** Extrémně rychlá (jeden takt). * **Nevýhoda:** Velká hardwarová náročnost, spotřeba.

E. Detekce přetečení (Overflow)

Výsledek se nevejde do bitů. * **Neznaménková čísla:** Přetečení, pokud C_{out} z MSB je 1. * **Znaménková čísla (doplňkový kód):** Přetečení, pokud sečteme dvě čísla stejného znaménka a výsledek má znaménko opačné. * **Hardwarová detekce:**

$$\text{Overflow Flag} = C_{in_{MSB}} \oplus C_{out_{MSB}}$$

(XOR přenosu do MSB a z MSB).

Praktické použití: Procesory (CPU), Mikrořadiče, DSP, FPGA akcelerátory, GPU.

Nejčastější chyby u zkoušky: Nezohlednění přenosu, záměna přetečení signed/unsigned, neznalost odčítání v doplňkovém kódu, špatná složitost CLA, neznalost kompromisů násobiček.

Paměťová pomůcka: Aritmetické obvody: Sčítačka je základ, dvojkový doplněk zjednodušuje, přetečení je past.

Typické zkouškové otázky (Zkrácené odpovědi)

Proč dvojkový doplněk?

Jednoznačná reprezentace nuly.

Zjednodušení aritmetiky: Sčítání i odčítání stejnou sčítačkou ($A - B = A + \bar{B} + 1$).

RCA vs CLA (rychlost, složitost)?

RCA: Sériové FA, pomalá ($O(n)$), nízká složitost.

CLA: Paralelní výpočet přenosů, rychlá ($O(\log n)$), vysoká složitost.

Odčítání $A - B$ pomocí sčítačky v doplňkovém kódu?

Rovnice: $A - B = A + \bar{B} + 1$.

Realizace: Bity B přes XOR řízené SUB, SUB jako C_{in} nejnižšího bitu.

Detekce přetečení (overflow) pro znaménková čísla?

Nastává, když (kladné+kladné=záporné) nebo (záporné+záporné=kladné).

Detekce: Overflow Flag = $C_{in_{MSB}} \oplus C_{out_{MSB}}$.

Sériová vs Paralelní násobička (rychlost, HW náročnost)?

Sériová: Pomalá (n taktů), nízká HW náročnost.

Paralelní: Velmi rychlá (1 takt), vysoká HW náročnost.

Shrnutí: Aritmetické obvody jsou základem CPU. Dvojkový doplněk zjednodušuje sčítání/odčítání. Sčítačky se liší rychlostí (RCA vs CLA). Násobičky mají kompromisy mezi rychlostí a HW (sériová vs paralelní). Klíčová je detekce přetečení, která se liší pro znaménková a neznaménková čísla.

37. Procesory, FPGA obvody (TAHÁK)

Klíčové pojmy

Procesor (CPU, MCU), instrukční soubor, registr (PC, SP), ALU, pipeline, cache, přerušení, konečný automat (FSM), FPGA, CLB/LE, LUT, klopný obvod (D-Flip-Flop), propojovací matice, I/O bloky, HDL (VHDL, Verilog), syntéza, bitstream, paralelismus, SoC.

A. Procesory (CPU / MCU)

Definice: Univerzální programovatelné sekvenční obvody. Sekvenčně čtou a provádějí strojové instrukce z paměti. **Architektura:** Řadič provádí **Fetch-Decode-Execute cyklus**: * **Fetch:** Načtení instrukce z paměti (adresa z **PC** - Program Counter). PC se inkrementuje. * **Decode:** Dekódování instrukce (operace, operandy). * **Execute:** Provedení operace (**ALU** - Aritmeticko-logická jednotka, paměť, I/O, skok). * **Výkon:** Zvyšován technikami jako **pipeline** (překrývání fází instrukcí) a **cache** (rychlá paměť pro často používaná data/instrukce). **Realizace FSM:** Softwarově. Stav v proměnné, logika `switch-case` v nekonečné smyčce. Striktně sekvenční, časová režie, omezený paralelismus. **Použití:** Běžné počítače, OS, embedded systémy (mikrokontroléry). **Typické chyby:** Ignorování pipeline/cache. Představa pravého paralelismu v rámci jednoho vlákna.

B. FPGA obvody (Field Programmable Gate Array)

Definice: Rekonfigurovatelná polovodičová součástka. Po výrobě neobsahuje pevnou funkci. Umožňuje implementovat vlastní, vysoce paralelní hardware. **Vnitřní architektura:** * **CLB / LE (Configurable Logic Blocks / Logic Elements):** Základní buňky. Obsahují: * **LUT (Look-Up Table):** Malá RAM pro kombinační logiku (pravdivostní tabulka). * **Klopný obvod (D-Flip-Flop):** Paměťový prvek pro sekvenční logiku (uložení stavu). * **Propojovací matice:** Síť programovatelných spínačů pro propojení logických bloků. * **I/O Bloky:** Rozhraní s fyzickými piny čipu. * (Moderní FPGA mají i DSP a paměťové bloky.) **Realizace FSM:** Hardwarově. LUT pro přechodovou/výstupní logiku, klopné obvody pro fyzické držení stavu. Masivní paralelismus, real-time, nízká latence. **Použití:** Zpracování signálů, měření/řízení, akcelerátory (kryptografie, AI), prototypování ASIC. **Typické chyby:** Tvzení, že FPGA "spouští program". Ignorování hodinových signálů a synchronizace.

C. Srovnání realizace automatů: Procesor vs. FPGA

Vlastnost	Procesor (CPU / MCU)	FPGA (Field Programmable
Povaha	Univerzální instrukční stroj	Rekonfigurovatelný hardware
Princip činnosti	Sekvenční vykonávání instrukcí	Paralelní implementace logiky
Realizace FSM	Softwarová (switch-case)	Hardwarová (LUT, klopné obv
Paralelismus	Omezený (pipeline, více jader)	Masivní, skutečně souběžné c
Latence	Vyšší	Velmi nízká
Flexibilita	Vysoká (snadná změna SW)	Střední (změna vyžaduje synt
Vývoj	Jednodušší (běžné jazyky)	Složitější (HDL, časování)

D. Jazyky pro popis hardwaru (HDL)

Popis: HDL (Hardware Description Language), např. **VHDL** nebo **Verilog**, definuje, jak se mají hradla a paměťové prvky propojit a chovat. Nejedná se o program, který se spouští. **Kompilační proces pro FPGA:** 1. **Syntéza:** HDL kód se přeloží do sítě logických hradel a klopných obvodů (Netlist). 2. **Implementation (Place & Route):** Mapování Netlistu na fyzické bloky FPGA a jejich propojení. 3. **Generování bitstreamu:** Vytvoření konfiguračního souboru, který se nahraje do FPGA a fyzicky vytvoří hardware. **Typické chyby:** Zaměňování HDL s běžným programovacím jazykem. Ignorování časování a kritické cesty.

E. Koncept SoC (System on Chip)

Definice a výhody: Integrovaný obvod, který kombinuje CPU a FPGA na jediném čipu. Nabízí flexibilitu softwaru (CPU) pro obecné úlohy a výkon/paralelismus hardwaru (FPGA) pro specifické akcelerované úlohy. **Příklady použití:** Vestavěné systémy, síťová zařízení, zpracování obrazu/video, akcelerace v datových centrech.

Typické zkouškové otázky

1. Jaký je rozdíl mezi CPU a FPGA?

CPU: Univerzální sekvenční stroj, vykonává instrukce (software). Flexibilní, ale operace jsou sekvenční. Chování se mění změnou softwaru.

FPGA: Rekonfigurovatelný hardware, fyzicky implementuje logiku (hardware). Umožňuje masivní paralelismus a nízkou latenci. Chování se mění rekonfigurací hardwaru (nový bitstream).

2. Co je pipeline a cache?

Pipeline (instrukční roura): Technika v CPU, která překrývá fáze vykonávání instrukcí (Fetch, Decode, Execute) pro zvýšení propustnosti procesoru.

Cache (vyrovnávací paměť): Malá, velmi rychlá paměť blízko CPU. Ukládá často používaná data/instrukce z hlavní paměti, čímž zkracuje dobu čekání procesoru a zvyšuje výkon.

3. Kdy zvolit FPGA místo procesoru?

FPGA je vhodnější pro úlohy vyžadující:

- * **Masivní paralelismus:** Mnoho souběžných operací (zpracování obrazu, DSP).
- * **Velmi nízkou latenci a determinismus:** Rychlá a předvídatelná odezva (řízení v reálném čase).
- * **Vysokou propustnost:** Pro objem dat nebo rychlost operací, kde CPU nestačí.
- * **Hardwarovou akceleraci:** Specifické algoritmy mapované na paralelní hardware.
- * **Flexibilitu hardwaru po výrobě:** Možnost změnit hardwarovou funkci čipu.